

CS 300 – Project One: Pseudocode and Runtime Analysis

1. Overview

ABCU's Computer Science advisors require a program that can print all courses in alphanumeric order and display the title and prerequisites for a selected course. To evaluate the best approach, pseudocode was developed using a vector, hash table, and binary search tree.

2. Shared loadCourses pseudocode

Because the file is not guaranteed to be sorted, a two-pass parsing routine is used:

```
loadCourses(structure, fileName)
    create empty Set courseIds

    first pass: store all courseNumbers in courseIds

    second pass:
        create Course object
        validate prerequisites using courseIds
        insert Course object into structure
end loadCourses
```

3. Menu Pseudocode

```
main()
    create empty data structure courses
    while user has not chosen Exit
        display options:
            1. Load Courses
            2. Display Course List
            3. Find Course
            9. Exit
        read user choice
        if choice = 1
            prompt for file name
            call loadCourses(courses, fileName)
        else if choice = 2
            call printCourseList(courses)
        else if choice = 3
            prompt for course number
            call searchCourse(courses, courseNumber)
        else if choice = 9
            exit program
```

```
    end if
  end while
end main
```

4. **printCourseList pseudocode**

```
printCourseList(structure)
  if structure is Vector
    sort and print
  else if Hash Table
    copy to temporary vector, sort, and print
  else if BST
    perform in-order traversal
  end printCourseList
```

5. **Runtime Evaluation (Worst-Case)**

Worst-case time for reading the file and creating course objects (n = number of courses):

Vector: $O(n)$ time, $O(n)$ space

Hash Table: $O(n^2)$ worst-case time, $O(n)$ space

Binary Search Tree: $O(n^2)$ worst-case time, $O(n)$ space

6. **Advantages and Disadvantages**

Vector is simple with $O(n)$ load time but requires $O(n)$ search and $O(n \log n)$ sorting. Hash table provides $O(1)$ average search but no natural ordering and $O(n^2)$ worst-case insertion. BST maintains sorted order and supports $O(\log n)$ average search but has $O(n^2)$ worst-case load if unbalanced.

7. **Recommendation**

Although the vector has the lowest load time, loading occurs only once. Advisors frequently search and print courses. The BST provides efficient search and maintains sorted order, making it the most appropriate choice.